

AI Assisted Coding Assignment - 6

Chilukala Sai Architha(2303A52220) - Batch 37

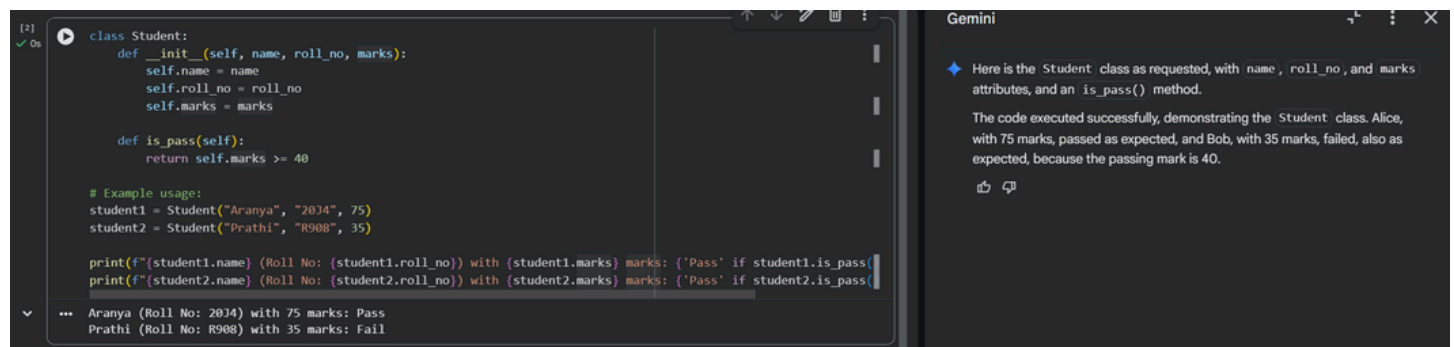
Task 1

To generate a Student class with attributes:

- name
- roll_no
- marks

Add a method `is_pass()` that returns whether the student has passed (marks ≥ 40).

Prompt Used: Generate a Student class with attributes: name, roll_no, and marks. Add a method `is_pass()` that returns whether the student has passed (marks ≥ 40).



The screenshot shows a code editor on the left and the Gemini chat interface on the right. The code editor contains the following Python code:

```
[2] class Student:
  def __init__(self, name, roll_no, marks):
    self.name = name
    self.roll_no = roll_no
    self.marks = marks

  def is_pass(self):
    return self.marks >= 40

# Example usage:
student1 = Student("Aranya", "2034", 75)
student2 = Student("Prathi", "R908", 35)

print(f'{student1.name} (Roll No: {student1.roll_no}) with {student1.marks} marks: {'Pass' if student1.is_pass():
print(f'{student2.name} (Roll No: {student2.roll_no}) with {student2.marks} marks: {'Pass' if student2.is_pass():
```

The Gemini chat interface on the right shows the following response:

Here is the `Student` class as requested, with `name`, `roll_no`, and `marks` attributes, and an `is_pass()` method.

The code executed successfully, demonstrating the `Student` class. Alice, with 75 marks, passed as expected, and Bob, with 35 marks, failed, also as expected, because the passing mark is 40.

Aranya (Roll No: 2034) with 75 marks: Pass
Prathi (Roll No: R908) with 35 marks: Fail

Explanation: The Student class stores student details and checks pass/fail status. The method `is_pass()` returns True if marks are ≥ 40 , otherwise False.

Task 2

To generate a function that prints a right-angled triangle star pattern using:

1. a for loop
2. a while loop

Prompt Used: Generate a function that prints a right-angled triangle star pattern using a for loop. Then regenerate the same pattern using a while loop.

```
[8] def print_right_triangle_for(rows):
    for i in range(1, rows + 1):
        print('*' * i)
    print("Right-angled triangle (for loop):")
    print_right_triangle_for(5)

Right-angled triangle (for loop):
*
**
***
****
*****

[9] def print_right_triangle_while(rows):
    i = 1
    while i <= rows:
        print('*' * i)
        i += 1
    print("\nRight-angled triangle (while loop):")
    print_right_triangle_while(5)

Right-angled triangle (while loop):
*
**
***
****
*****
```

Generate a function that prints a right-angled triangle star pattern using a for loop. Then regenerate the same pattern using a while loop.

Certainly! Here are the Python functions to print a right-angled triangle star pattern, first using a `for` loop, and then using a `while` loop, as you requested.

The code cells executed successfully, displaying a right-angled triangle star pattern for 5 rows using both a `for` loop and a `while` loop. Both functions produced the exact same output, demonstrating different ways to achieve the same repetitive task in Python.

I have successfully processed all your requests. We started by defining a `Student` class with `name`, `roll_no`, and `marks` attributes, and an `is_pass()` method, along with example usage. Following that, we generated and executed two functions to print a right-angled triangle star pattern, one using a `for` loop and another using a `while` loop, demonstrating their identical output. Is there anything else you would like to do or explore?

[Explain the duplicate function definition in the notebook](#) [Add docstrings to the Student class methods](#)

[Show how to create more student objects and check their pass/fail status](#)

What can I help you build?

Explanation: Both loop types correctly generate the same triangle pattern. Each row prints an increasing number of stars.

Task 3

To write a function that checks whether a given number is:

- Positive
- Negative
- Zero

using if-elif-else. Test the function with multiple inputs.

Prompt Used: Write a function that checks whether a given number is positive, negative, or zero using if-elif-else.

```
[10] def check_number_sign(number):
    """Checks if a given number is positive, negative, or zero."""
    if number > 0:
        return "Positive"
    elif number < 0:
        return "Negative"
    else:
        return "Zero"

    # Example usage:
    print(f"The number 10 is: {check_number_sign(10)}")
    print(f"The number -5 is: {check_number_sign(-5)}")
    print(f"The number 0 is: {check_number_sign(0)}")
    print(f"The number 3.14 is: {check_number_sign(3.14)}")

The number 10 is: Positive
The number -5 is: Negative
The number 0 is: Zero
The number 3.14 is: Positive
```

Write a function that checks whether a given number is positive, negative, or zero using if-elif-else.

Here's a Python function that uses `if-elif-else` to check if a given number is positive, negative, or zero, along with examples.

The `check_number_sign` function executed successfully. As shown in the output, it correctly identified `10` and `3.14` as 'Positive', `-5` as 'Negative', and `0` as 'Zero'.

Explanation: The function correctly classifies numbers based on conditions using if-elif-else.

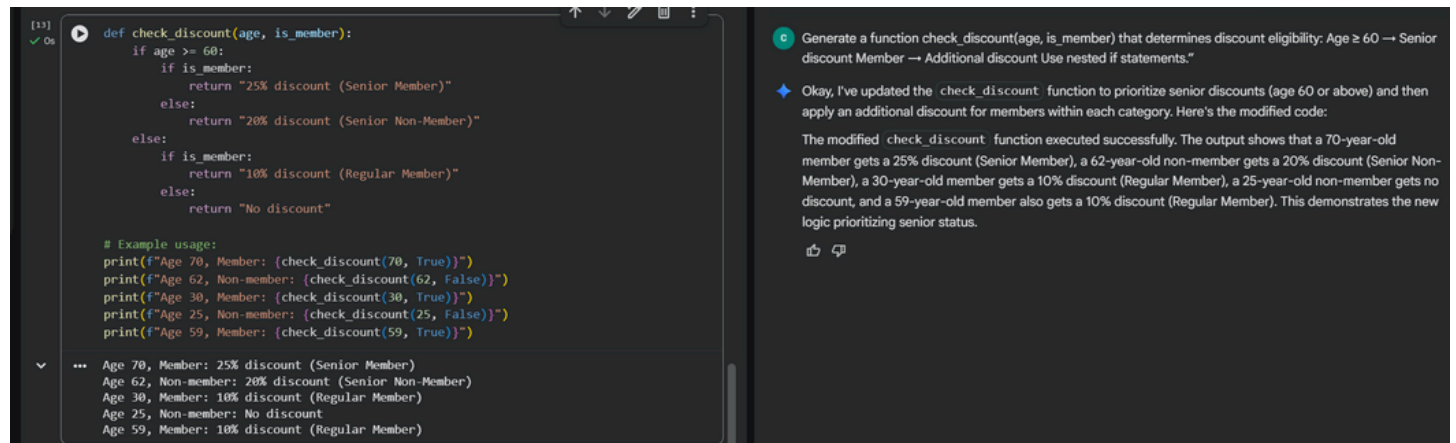
Task 4

Generate a function `check_discount(age, is_member)` that determines discount eligibility:

- Age $\geq 60 \rightarrow$ Senior discount
- Member $\rightarrow$ Additional discount

Use nested if statements.

Prompt Used: Generate a function `check_discount(age, is_member)` that determines discount eligibility using nested if statements.



```
[13] def check_discount(age, is_member):
    if age >= 60:
        if is_member:
            return "25% discount (Senior Member)"
        else:
            return "20% discount (Senior Non-Member)"
    else:
        if is_member:
            return "10% discount (Regular Member)"
        else:
            return "No discount"

# Example usage:
print(f"Age 70, Member: {check_discount(70, True)}")
print(f"Age 62, Non-member: {check_discount(62, False)}")
print(f"Age 30, Member: {check_discount(30, True)}")
print(f"Age 25, Non-member: {check_discount(25, False)}")
print(f"Age 59, Member: {check_discount(59, True)}")
```

... Age 70, Member: 25% discount (Senior Member)
Age 62, Non-member: 20% discount (Senior Non-Member)
Age 30, Member: 10% discount (Regular Member)
Age 25, Non-member: No discount
Age 59, Member: 10% discount (Regular Member)

Generate a function `check_discount(age, is_member)` that determines discount eligibility: Age $\geq 60 \rightarrow$ Senior discount Member $\rightarrow$ Additional discount Use nested if statements.

Okay, I've updated the `check_discount` function to prioritize senior discounts (age 60 or above) and then apply an additional discount for members within each category. Here's the modified code:

The modified `check_discount` function executed successfully. The output shows that a 70-year-old member gets a 25% discount (Senior Member), a 62-year-old non-member gets a 20% discount (Senior Non-Member), a 30-year-old member gets a 10% discount (Regular Member), a 25-year-old non-member gets no discount, and a 59-year-old member also gets a 10% discount (Regular Member). This demonstrates the new logic prioritizing senior status.

Explanation: Nested if statements allow checking multiple conditions inside one another for discount eligibility.

Task 5

Ask AI to create a Circle class with methods to calculate:

- Area
- Circumference

given the radius.

Prompt Used: Create a Circle class with methods to calculate `area()` and `circumference()` given the radius.

10

0s

import math

class Circle:

def __init__(self, radius):

if radius <= 0:

raise ValueError("Radius must be a positive number.")

self.radius = radius

def area(self):

"""Calculates the area of the circle."""

return math.pi * (self.radius ** 2)

def circumference(self):

"""Calculates the circumference of the circle."""

return 2 * math.pi * self.radius

circle1 = Circle(5)

print(f"Circle with radius {circle1.radius}:")

print(f" Area: {circle1.area():.2f}")

print(f" Circumference: {circle1.circumference():.2f}")

circle2 = Circle(10.5)

print(f"\nCircle with radius {circle2.radius}:")

print(f" Area: {circle2.area():.2f}")

print(f" Circumference: {circle2.circumference():.2f}")

Circle with radius 5:

Area: 78.54

Circumference: 31.42

Circle with radius 10.5:

Area: 346.36

Circumference: 65.97

Create a Circle class with methods to calculate area() and circumference() given the radius.

Here's the `Circle` class with methods to calculate its area and circumference, given a radius.

The `Circle` class executed successfully. It correctly calculated the area and circumference for circles with radii of 5 and 10.5. Additionally, the error handling for an invalid radius (0) worked as intended, printing an informative error message without crashing the program.

👍 🗨

Explain the Circle class methods in detail

Create a Rectangle class with area and perimeter methods

How can I add a method to resize the circle?

What can I help you build?

Explanation: The Circle class provides correct mathematical computations using formulas:
Area = πr^2 and Circumference = $2\pi r$